ARUN KUMAR

SOFTWARE DEVELOPMENT | C++ & PYTHON SYSTEMS ENGINEERING | LINUX · DISTRIBUTED SYSTEMS · AI AGENT ENGINEERING

Thanjavur, Tamil Nadu, India | +91 78068 63734 | arunanthony47@gmail.com
linkedin.com/in/arun-kumar-619358210 | github.com/ELARION12

PROFESSIONAL SUMMARY

Software engineer with **3+ years** of hands-on experience building and validating systems-level software in **C++, Python, and Linux** spanning distributed systems, real-time embedded control, OS internals, and network programming. Conducts in-depth code review, architecture assessment, and cross-platform benchmarking on production-grade software, working daily with modern engineering and AI-assisted development toolchains (Git, GDB, GCC, Claude Code, Cursor, GitHub Copilot). Built an open-source AI agent orchestration runtime engineered from scratch in C, C++, and Python. Backed by hardware/firmware engineering bare-metal ARM Cortex-M4 development, RTOS, and CAN bus bringing full-stack engineering judgment from silicon to software.

TECHNICAL SKILLS

Programming Languages	C++(17/20/23), C(11), Python3, Embedded C, CUDA C++
Systems & OS Engineering	Linux System Programming, OS Internals, Multithreading & Concurrency, Mutexes & Semaphores, RTOS, Memory Management (Arena Allocators), POSIX API, TCP/IP Networking, Network Protocol Stack, System Debugging, Software Testing & Validation
AI / LLM & Agent Engineering	CUDA GPU Acceleration, Pybind11, SIMD Verctorization(AVX2), FastMCP Tool Orchestration, Local LLM Integration (Ollama), Claude Code, Gemini CLI, Cursor, Windsurf, GitHub Copilot
Embedded & Hardware	ARM Cortex-M4, STM32 Peripherals, Bare-Metal Register Programming, SPI, I2C, LTDC, SDRAM, CAN Protocol, BIOS Firmware, Chip-Level Diagnostics
Tools & Infrastructure	GCC Toolchain, GDB, CMake, Valgrind, AddressSanitizer (ASan), Keil MDK, STM32CubeIDE, Docker, NumPy, Pandas, Linux CLI, Git

TECHNICAL EXPERIENCE

Software Engineering Consultant — Code Review, Architecture & Benchmarking (Freelance) [Jun 2025 – Present]
Ecademic Tube — Online Technical Education Platform | Remote

- Reviewed 150+ engineering software solutions in C++, Python, and Linux systems — evaluating correctness, edge cases, performance bottlenecks, and implementation quality against professional engineering standards.
- Assessed systems-level code for concurrency bugs, race conditions, memory management issues, and architectural tradeoffs across OS, networking, and infrastructure software domains.
- Benchmarked and compared coding-agent outputs (Claude Code, Gemini CLI, Cursor, GitHub Copilot, ChatGPT/Codex) to quantify differences in correctness, scalability, and engineering tradeoffs on complex systems tasks.
- Delivered structured technical feedback on algorithm design, software architecture, and debugging methodology, supporting software-quality benchmarking pipelines.
- Authored technical documentation on Linux programming patterns, concurrency debugging workflows, and systems-level problem solving.
- Reviewed and annotated embedded code in MATLAB, Verilog, and Embedded C for functional and logical correctness.

Hardware & Firmware Specialist (Self-Employed) [May 2022 – Present]
Remote / On-site

- Performed chip-level diagnostics and component-level failure analysis on laptop/desktop hardware — tracing faults through power delivery circuits, memory subsystems, and peripheral interfaces using multimeters and oscilloscopes.
- Executed BIOS firmware programming, editing, and optimization — flashing updated images and resolving boot-sequence failures caused by corrupted or misconfigured firmware.

- Recovered data and restored functionality across hardware failure scenarios using structured fault-isolation techniques.
- Built practical expertise in hardware-software interfaces and signal-level debugging that directly underpins bare-metal embedded systems work.

Embedded Systems Engineer — Training & Project [May 2024 – Dec 2024]
Vector India, Chennai

- Completed an intensive 6-month professional program in Embedded Systems, Linux Programming, RTOS, C++, and TCP/IP networking.
- Studied TCP/IP protocol stack fundamentals — socket programming, transport-layer behavior, and connection management — building foundational network-systems knowledge.
- Designed and implemented a distributed real-time embedded control system (Body Control Module) with multi-node CAN bus communication, addressing bus arbitration, message scheduling, and fault handling under concurrent load.
- Developed state-machine-based control logic in Embedded C/C++ for automotive subsystems, coordinating distributed node behavior under RTOS scheduling constraints.

TECHNICAL PROJECTS

IronAgent — Bare-Metal AI Agent Orchestrator (Open Source) [Jul 2026]
Differentiator: Custom 2ns AVX2-aligned memory arena and O(1) context pruning — a fully software AI agent runtime with zero embedded/hardware dependency.

- Built a cross-language AI agent runtime — a Python API bridged via Pybind11 to a custom C/C++ execution core — implementing a Think → Plan → Act orchestration loop.
- Designed a custom AVX2-aligned memory arena allocator (~2ns per allocation) and a std::deque-based O(1) context-sliding-window that prunes context in under 0.5ms, removing garbage-collection pauses from long-running agent sessions.
- Benchmarked orchestration throughput and memory footprint against LangChain (208MB vs. 424MB cold-start RAM) and documented an FFI/Pybind11 boundary bottleneck, defining a zero-copy roadmap for v1.1.
- Implemented native FastMCP tool routing and local LLM (Ollama) integration for sandboxed, privacy-first agent execution with no cloud API dependency.

Languages & Tools: Python, C++, C11, CMake, Pybind11, pthreads, AVX2
AI/LLM Technologies: FastMCP Tool Orchestration, Ollama (Local LLM Runtime), Agentic Think-Plan-Act Architecture
Outcome: Delivered an open-source, fully software AI agent runtime demonstrating systems-level mastery of memory management, concurrency, and cross-language architecture applied to modern agentic AI engineering.

Low-Level Clock & Peripherals — STM32F429I-DISC1 [May 2024]
Differentiator: Bare-metal ARM clock-tree (RCC/PLL) configuration from first principles — zero HAL or middleware.

- Configured ARM Cortex-M4 core clock architecture (RCC/PLL) from scratch to drive the processor at its maximum operating frequency.
- Developed bare-metal peripheral drivers via direct register manipulation (Mode, Speed, Pull-up/Pull-down) with no HAL abstraction layer.
- Designed high-precision timing routines using the ARM SysTick hardware timer for millisecond-accurate event scheduling.
- Validated register-state changes and signal timing using the ST-Link debugger and logic analyzer.

Languages & Tools: Embedded C, ARM Cortex-M4, SysTick, RCC, PLL, GPIO, ST-Link Debugger
Outcome: Delivered a fully verified bare-metal clock and peripheral initialization layer, demonstrating register-level mastery of ARM Cortex-M4 architecture.

Bare-Metal Battleship Game Engine — STM32F429I-DISC1 [Jun 2025]
Differentiator: Predictive AI opponent driven entirely by hardware RNG — no software pseudo-random dependency, no RTOS/HAL.

- Architected a bare-metal FSM-based game engine managing menus, grid rendering, turn logic, and win/loss evaluation with no RTOS, middleware, or HAL.
- Engineered a predictive “Hunt and Target” AI opponent interfacing directly with the hardware RNG peripheral.
- Built low-level SPI/I2C drivers from scratch for the ILI9341 LCD and STMPE811 touchscreen controller.
- Diagnosed and resolved SDRAM initialization issues causing blank-display failures, and calibrated touch-input coordinate mapping.

Languages & Tools: Embedded C, ARM Cortex-M4, SPI, I2C, LTDC, SDRAM, Hardware RNG, Keil MDK / STM32CubeIDE

Outcome: Delivered a complete standalone embedded game engine with touchscreen input, hardware-accelerated LCD output, and intelligent AI opponent logic.

Body Control Module — CAN Bus Distributed System [Vector India, 2024]

Differentiator: Multi-node real-time CAN arbitration and fault handling under concurrent load with strict RTOS timing guarantees.

- Designed a distributed real-time control architecture with multi-node CAN bus communication, addressing bus arbitration, message scheduling, and fault handling under concurrent load.
- Developed state-machine-based control logic in Embedded C/C++ for automotive subsystems (lighting, wiper, power window, door lock).
- Configured CAN message-handling and filtering strategies to minimize bus traffic and improve throughput.
- Validated real-time responsiveness and robustness through systematic debugging across multiple operational scenarios.

Languages & Tools: Embedded C, C++, CAN Protocol, RTOS, GCC Toolchain, GDB

Outcome: Delivered a stable multi-node distributed control solution demonstrating real-time inter-node communication and fault-tolerant state management.

IoT-Based Two-Way Communicable Doorbell System [Dec 2021 – Apr 2022]

Differentiator: Real-time deep-learning inference on edge hardware, purpose-built for accessibility.

- Built a smart doorbell integrating a deep-learning object detection pipeline in Python, processing real-time visitor data on a Raspberry Pi.
- Implemented automated cross-platform notification logic via IFTTT webhooks for instant Android/iOS alerts.
- Designed the system specifically for deaf users, prioritizing reliable visual/push alerts over audio cues.

Languages & Tools: Raspberry Pi, Python, NumPy, Pandas, Deep Learning, Object Detection, IFTTT Webhooks

Outcome: Delivered a working assistive IoT system demonstrating edge ML inference and cloud-connected notification architecture.

EDUCATION

B.E., Electronics and Communication Engineering — CGPA: 8.26 [2018 – 2022]
Kings College of Engineering (Affiliated to Anna University), Chennai

Embedded & Linux Development with C++ & RTOS — Professional Certification [May 2024 – Dec 2024]
Vector India, Chennai | 6-Month Training

LANGUAGES

English (Fluent) | Tamil (Native)